

Behavioral Biometric Authentication System

Mouse Dynamics · Keystroke Analysis · Touchscreen Gestures

AUTHORS

Bouchrit Maroua · Gougam Lina · Gaceb Riham Bahia

Academic Year 2025–2026 · Module: Computer & Network Security

LIVE DEMO (Touchscreen): <https://spontaneous-brioche-76fba3.netlify.app/>

Abstract

This report presents a multi-modal behavioral biometric authentication system that continuously verifies user identity through three complementary interaction channels: mouse dynamics, keystroke dynamics, and touchscreen gestures. Unlike static credential-based authentication, behavioral biometrics captures subconscious motor patterns — timing rhythms, movement trajectories, and pressure profiles — that are deeply personal and extraordinarily difficult to replicate under impersonation. The system employs three machine learning approaches: Isolation Forest and One-Class SVM for unsupervised one-class anomaly detection (mouse and touchscreen), and a per-user binary Random Forest for supervised classification (keystroke). Evaluated across 14 users (196 sessions) for mouse dynamics, 718 users from the Aalto Desktop dataset plus 99 from KeyRecs for keystroke, and 5 users (912,133 touch events) for touchscreen gestures, the system achieved mean Equal Error Rates of 33.19%, 12.87%, and 28.6% respectively. The combined multi-factor architecture — requiring behavioral scores above 0.60 for both keystroke and mouse channels — raises the security bar substantially beyond single-factor approaches.

33.19%
Mouse EER

12.87%
Keystroke EER

28.60%
Touchscreen EER

718
Users (Keystroke)

Table of Contents

1. Introduction

1.1 Motivation

1.2 Problem Statement

1.3 Project Objectives

2. System Architecture & Authentication Logic

3. Datasets

3.1 Mouse & Keystroke Dataset

3.2 Keystroke Benchmark Datasets

3.3 Touchscreen Dataset

4. Machine Learning Models

4.1 Isolation Forest

4.2 One-Class SVM

4.3 Per-User Random Forest

4.4 Evaluation Metrics

5. Part I — Mouse Dynamics

6. Part II — Keystroke Dynamics

7. Part III — Touchscreen Gesture Dynamics

8. Cross-Modality Results Summary

9. Real-Time Authentication System

10. Limitations & Future Work

11. Conclusion

References

1 Introduction

1.1 Motivation

The vast majority of authentication systems in production today rely on shared secrets: passwords, PINs, and security tokens. These mechanisms share a fundamental structural weakness — the secret can be stolen, guessed, or phished without the attacker needing to physically resemble or behaviorally mimic the legitimate user. Once credentials are compromised, traditional systems grant unrestricted access with no further verification.

Behavioral biometrics addresses this gap by transforming the session itself into a continuous authentication signal. Every keyboard interaction, every mouse movement, every touchscreen stroke carries a latent behavioral fingerprint shaped by decades of individual motor learning.

1.2 Problem Statement

Can the natural, unconscious interaction patterns of a user — their typing rhythm, mouse kinematics, and gesture dynamics — serve as a reliable, continuous authentication factor capable of detecting impostors even when those impostors possess the correct credentials?

1.3 Project Objectives

- 0 Capture and process raw browser-level interaction events (mouse, keyboard, touch) into discriminative
1 feature representations.
- 0 Train per-user anomaly detection and classification models using Isolation Forest, One-Class SVM, and
2 Random Forest.
- 0 Evaluate system performance using FAR, FRR, EER, and AUC-ROC across realistic impostor
3 scenarios.
- 0 Integrate all three modalities into a live, deployable web application enabling real-time continuous
4 authentication.

2 System Architecture & Authentication Logic

The system is organized across three layers. The **Data Layer** collects raw events from the browser using JavaScript event listeners (`mousemove`, `mousedown`, `mouseup`, `keydown`, `keyup`, `touchstart`, `touchmove`, `touchend`) and transmits timestamped event streams to a Flask backend via HTTP POST. The **ML Layer** processes incoming event streams into fixed-length feature vectors, manages per-user model storage, and produces normalized authentication scores in $[0, 1]$. The **Interface Layer** presents a four-tab web application (Enroll, Authenticate, Statistics, How It Works) that visualizes real-time scores and authentication decisions.

Authentication is based on two behavioral conditions that must both be satisfied simultaneously:

Factor	Weight	Minimum Threshold
Keystroke dynamics score	50%	≥ 0.60
Mouse dynamics score	50%	≥ 0.60

The combined weighted score must exceed 0.60 and neither behavioral component may fall below its threshold. This ensures that an attacker who partially replicates one modality cannot compensate for failure in the other.

3 Datasets

3.1 Mouse & Keystroke Dataset — EVTRACKTRACK.csv

This in-house dataset was captured via a custom browser instrumentation tool recording six event types: `mousemove`, `mousedown`, `mouseup`, `click`, `keydown`, and `keyup`. Each event record contains a Unix millisecond timestamp, event type code, and for mouse events the (x, y) screen coordinates; for keyboard events, the raw key code. The AZERTY layout was used, with observed key codes including 71(G), 30(A), 38(L), 83(S), 94(^), 106(J), 85(U), 87(W), 89(Y), and 100(D).

Metric	Value
Total records	143,115
Unique users	14
Unique sessions	196
Date range	Feb 2020 - Jan 2024

3.2 Keystroke Benchmark Datasets

Aalto University Desktop Dataset is one of the largest public keystroke corpora in existence, comprising 136 million keystrokes from 168,000 participants who typed English sentences via a browser-hosted website. Each participant typed up to 15 different sentences across multiple sessions, with every key press and release captured at millisecond precision using JavaScript `keydown` / `keyup` events. Because loading all 168,000 user files simultaneously would require over 100 GB of RAM, a two-stage smart sampling strategy was applied: first, a lightweight scan reading only user ID and session ID columns (approximately 14 minutes, near-zero memory), then selective loading of the top 800 users by session count, resulting in approximately 150 MB of working data.

KeyRecs Free-Text Dataset was collected at Polytechnic of Porto in 2023 from 99 participants across 20 nationalities. Unlike Aalto, participants typed freely rather than from fixed prompts, and the dataset stores pre-computed inter-key timing values (Down-Down and Up-Down intervals) rather than raw timestamps. The dataset includes demographic attributes (age, gender, handedness, nationality), enabling auxiliary behavioral analysis.

Property	Aalto Desktop	KeyRecs Free-Text
Total users available	168,000	99
Users sampled / used	718 (after cleaning)	99 (all)

Sessions per user	15 (mean)	2 (mean)
Raw keystroke rows	578,803	562,583
Collection method	Browser keydown/keyup	Pre-computed digraph timings

3.3 Touchscreen Dataset

Touchscreen data was captured from 5 users (IDs 0-4) with approximately 41 sessions each, producing 912,133 raw touch events and 232 extracted gestures. Each event is tagged with one of three types: 0=Move (finger dragging), 1=Touch Down (initial contact), 2=Touch Up (release).

4 Machine Learning Models

4.1 Isolation Forest — Theory & Configuration

Isolation Forest is an ensemble anomaly detection algorithm that exploits the geometric property that anomalous points require fewer random binary partitions to be isolated from the rest of the data. The algorithm builds `n_estimators` independent random isolation trees, each constructed by recursively selecting a random feature and a random split value within that feature's observed range until each data point occupies its own leaf node. The path length $h(x)$ — the number of splits needed to isolate point x — is the core measurement: genuine data points embedded in dense behavioral clusters require many splits; anomalous points (impostors) are isolated quickly.

The anomaly score normalizes path length against the expected path length for randomly distributed data:

$$s(x, n) = 2^{(-E[h(x)] / c(n))}$$

where $E[h(x)]$ is the mean path length across all trees and $c(n) = 2 \cdot H(n-1) - (2(n-1)/n)$ is the average path length of an unsuccessful binary search tree over n samples (H = harmonic number). Scores near 1.0 indicate strong anomalies; scores near 0.5 indicate normal behavior; scores below 0.5 are very likely genuine.

Configuration

Modality	<code>n_estimators</code>	Contamination	Threshold
Mouse dynamics	100	0.05-0.10	10th percentile of training scores
Touchscreen	200	0.1	70/30 train/test split

The contamination hyperparameter is critical: it tells the algorithm what fraction of the training data is expected to be anomalous. Setting it too low creates an overly permissive boundary (high FAR); too high creates an overly tight boundary (high FRR). Values of 0.05-0.10 reflect the assumption that roughly 5-10% of a typical user's own sessions may be outliers due to fatigue, distraction, or unusual context.

4.2 One-Class SVM — Theory & Configuration

One-Class SVM (OC-SVM), introduced by Scholkopf et al. (2001), learns a hypersphere in kernel space that encloses the genuine user's behavioral data while minimizing the volume of enclosed space. The optimization objective is:

Minimize: $(1/2)||w||^2 + (1/\nu \cdot n) \cdot \sum(x_{i_i}) - \rho$

Subject to: $(w \cdot \Phi(x_{i_i})) \geq \rho - x_{i_i}, \quad x_{i_i} \geq 0$

where ν is in $(0,1]$ is the upper bound on the fraction of outliers (analogous to contamination), ρ is the decision threshold offset, and Φ is the kernel mapping. A new sample x is classified as genuine if $f(x) = \text{sign}((w \cdot \Phi(x)) - \rho) > 0$.

RBF kernel was used: $K(x, x_i) = \exp(-\gamma \cdot ||x - x_i||^2)$, with $\gamma = \text{'scale'}$ (automatically set to $1/(n_{\text{features}} \cdot \text{Var}(X))$) and $\nu = 0.1$. The RBF kernel is ideal for behavioral data because it can capture nonlinear, non-convex cluster shapes — user behavior rarely occupies a convex region in feature space.

The key advantage of OC-SVM over Isolation Forest is that it constructs an explicit decision boundary defined by support vectors (the genuine samples closest to the boundary), making it more interpretable and robust in low-sample regimes. Its weakness is computational cost: training scales as $O(n^2)$ to $O(n^3)$ with number of samples, making it unsuitable for very large enrollment datasets.

4.3 Per-User Random Forest — Theory & Configuration

The per-user Random Forest departs from the one-class paradigm by training a binary classifier that explicitly contrasts the genuine user's behavior against a representative sample of impostor behavior drawn from all other users. For each of the 718 enrolled users, one independent Random Forest was trained with the following setup:

Positive class: all genuine windows from the target user.

Negative class: randomly sampled windows from other users at a 1:3 genuine-to-impostor ratio.

Architecture: 100 trees, max depth 10, `class_weight='balanced'`.

Split criterion: Gini impurity; $\sqrt{n_{\text{features}}}$ approx 4-5 features per split from the 21-dimensional space.

Each tree votes genuine or impostor, and the final score is the fraction of trees voting genuine (probability estimate). The decision threshold is set at 0.60, calibrated to balance FAR and FRR.

Why RF outperforms OC-SVM: Seeing impostor examples during training allows the forest to learn not just 'what the genuine user looks like' but also 'what the boundary between this user and others looks like.' In the 21-dimensional feature space, many users share large overlapping regions (e.g., similar mean typing speeds), but diverge sharply in other dimensions (e.g., dwell time distribution skewness). The RF can automatically identify and weight these discriminative dimensions per user, whereas OC-SVM must infer them from the shape of the genuine cluster alone.

4.4 Evaluation Metrics

Metric	Formula	Interpretation
FAR	$FP / (FP + TN)$	Fraction of impostor sessions wrongly accepted
FRR	$FN / (FN + TP)$	Fraction of genuine sessions wrongly rejected
EER	$FAR = FRR$ at optimal threshold	Lower = better; primary comparison metric
AUC-ROC	Area under ROC curve	1.0 = perfect ranking; 0.5 = random

The EER is computed by sweeping the decision threshold from 0 to 1 and finding the crossing point of the FAR and FRR curves. It is threshold-independent — a low EER means the score distributions of genuine and impostor users are well-separated regardless of the operating point chosen.

PART I Mouse Dynamics

5.1 Feature Engineering — 21-Dimensional Behavioral Vector

Raw mouse event streams are noisy, variable-length, and timestamp-driven. The feature engineering pipeline segments each session into movement strokes (sequences of `mousemove` events bounded by `mousedown` and `mouseup` events) and computes 21 statistical aggregates:

Velocity Features — *derived as $speed = Euclidean_distance(p_t, p_{t-1}) / \Delta_t$ (pixels/ms)*

Feature	Description
speed_mean	Average movement velocity across the session
speed_std	Velocity consistency — high std indicates erratic movement
speed_25, speed_75	Interquartile range capturing typical slow/fast phases

Acceleration Features — *derived as $accel = \Delta_{speed} / \Delta_t$*

Feature	Description
accel_mean	Average acceleration magnitude
accel_std	Jerk consistency — smooth vs. impulsive movement style
accel_25, accel_75	Acceleration percentile range

Curvature Features — *derived as $angle = \arctan2(\Delta_y, \Delta_x)$ between consecutive movement vectors*

Feature	Description
angle_mean	Average directional change — high values indicate meandering paths
angle_std	Variability in direction; consistent users show low std
angle_25, angle_75	Typical angular range during navigation

Spatial Features

Feature	Description
x_range, y_range	Horizontal and vertical coverage of the session
total_distance	Cumulative path length in pixels
avg_speed_overall	Total distance / session duration

Temporal & Interaction Features

Feature	Description
session_duration	Total session length in seconds
avg_pause	Mean idle time between movement bursts (ms)
long_pauses	Count of pauses > 500ms — captures thinking/hesitation patterns
click_count	Number of click events — task engagement intensity
move_count	Total number of mousemove events recorded

5.2 Anomaly Scoring & Authentication Decision

An independent Isolation Forest model is trained per user on their enrollment sessions. At authentication time, the incoming session is processed into a feature vector and scored. The raw Isolation Forest score (anomaly score in [-1, 1]) is linearly mapped to [0, 1] for the UI. The mouse component contributes 35% of the weighted authentication score. Access is granted only if the normalized score ≥ 0.60 .

5.3 Results & Analysis

The mouse dynamics module achieved an EER of **33.19%** on the 14-user dataset. This result warrants careful interpretation:

The primary driver of high EER is behavioral overlap in a small cohort. With only 14 users, several individuals share similar mouse kinematic profiles — particularly `speed_mean` and `total_distance` — making their behavioral clouds partially overlap in the 21-dimensional feature space. The Isolation Forest, trained only on genuine data, cannot learn where one user’s cloud ends and another’s begins; it can only learn the shape of the genuine user’s distribution. In a 1-vs-13 evaluation scenario (one genuine user tested against all 13 others acting as impostors), this structural overlap generates substantial false acceptances.

A secondary factor is session sparsity: some users have as few as 3 training sessions, providing the Isolation Forest with an extremely thin sample of the genuine behavioral distribution. With 3 sessions mapped to ~3 feature vectors, the model essentially memorizes the training points rather than learning a generalizable envelope, leading to tight boundaries and high FRR.

In live registration mode with fresh enrollment (Mode 2, 3 sessions enrolled on the day), performance improves substantially because the within-day behavioral variance is lower and the model boundary fits more tightly around the genuine cluster.

PART II Keystroke Dynamics

6.1 Data Preparation & Cleaning Pipeline

Before feature extraction, raw Aalto data undergoes a 7-step cleaning pipeline designed to remove physiologically impossible values and non-behavioral keystrokes:

Step	Operation	Details
1	Null removal	Rows with missing PRESS_TIME or RELEASE_TIME (70 rows — hardware errors).
2	Temporal sorting	Reorder keystrokes by PRESS_TIME within each session (no rows removed — structural fix).
3	Dwell filtering	Remove dwell times outside [0, 500ms] — 3,272 rows removed. Physical impossibility: human fingers cannot hold a key for more than ~500ms in normal typing.
4	Flight time filtering	Remove FLIGHT times outside [-300ms, 2000ms] — 3,924 rows. Negative flight times (key overlap) up to -300ms are physiologically plausible for fast typists using rollover; beyond this, they indicate clock errors.
5	DD time filtering	Remove Down-Down times outside [0, 2000ms] — 5,261 rows. DD times above 2s indicate an inter-word pause that should not be attributed to a single digraph rhythm.
6	Special key removal	Remove non-printable keys (Shift, Ctrl, Alt, Backspace, Enter) — 100,634 rows.
7	Short session removal	Remove sessions with fewer than 20 valid keystrokes — 8,924 rows.

Result: 578,803 raw rows -> 456,718 clean rows. 795 users loaded -> 718 retained (77 dropped because post-cleaning session count fell below the 3-session minimum for training).

6.2 Derived Timing Features

Feature	Formula	Description
---------	---------	-------------

DWELL (Hold Time)	RELEASE_TIME PRESS_TIME	-	Duration a key is physically depressed. Population mean: 109ms, std: 36ms. Dwell time is the most person-specific feature: fast touch-typists show consistently low dwell times (55-80ms), while hunt-and-peck typists show high, variable dwell times (150-250ms).
FLIGHT (Inter-key Gap)	PRESS_TIME[k+1] RELEASE_TIME[k]	-	Time between releasing one key and pressing the next. Population mean: 116ms. Negative flight times (key overlap) are common in expert typists and are a highly distinctive behavioral signal.
DD (Digraph Interval)	PRESS_TIME[k+1] PRESS_TIME[k]	-	Time between consecutive key presses, regardless of release. Mean: 225ms. This is the most commonly cited feature in keystroke dynamics literature because it is robust to minor variations in key-release timing.

6.3 Sliding Window Feature Extraction

Rather than operating on individual keystrokes, the system aggregates timing measurements over windows of 20 consecutive keystrokes with a step size of 10 (50% overlap between adjacent windows). This windowing strategy provides three key benefits:

- > **Noise reduction:** individual keystroke timings are noisy; averaging over 20 keystrokes produces a stable behavioral summary.
- > **Continuous scoring:** overlapping windows enable near-real-time authentication updates (a new score every ~10 keystrokes).
- > **Session-length independence:** the same 21-feature vector is produced regardless of how long the session is.

For each window, 21 statistical features are computed across DWELL, FLIGHT, and DD: 5 statistics per timing column (mean, std, median, IQR, skewness); 3 coefficient of variation features; 3 rhythm features (typing speed, pause rate, fast rate).

The train/test split is performed at the **session level**, not the window level. All windows from a given session go entirely into either train or test, never both. This is critical: window-level splitting would leak behavioral context across boundaries and produce optimistically biased EER estimates.

Yield: Aalto produced 30,930 windows from 718 users (~43 windows per user); KeyRecs produced 54,761 windows from 99 users (~553 windows per user due to longer free-text sessions).

6.4 Model Training & Evaluation

One-Class SVM (Baseline): For each of the 718 Aalto users, an OC-SVM is trained on that user's genuine windows only. At authentication time, a new window is scored as genuine (inside the hypersphere boundary) or anomaly (outside). The primary advantage is zero impostor data requirement — realistic for cold-start

enrollment.

Per-User Random Forest (Primary): For each user, a binary RF classifier is trained with genuine windows (positive class) and a random sample of windows from other users at a 1:3 ratio (negative class). The negative sampling is stratified across all other users to ensure the impostor representation covers the full behavioral diversity of the population.

Feature importance analysis (averaged across all 718 per-user RF models) places dwell-related features in 6 of the top 10 positions, with `dwell_mean` and `dwell_std` ranking first and second. This is consistent with keystroke biometrics theory: dwell time is driven by the neuromuscular control of individual fingers and is largely independent of what is being typed.

6.5 Results

Model	Dataset	Mean EER	Mean AUC-ROC	Mean FAR	Mean FRR	EER < 10%
One-Class SVM	Aalto Desktop	18.14%	0.8864	18.23%	23.22%	139/717
Per-User RF	Aalto Desktop	12.87%	0.9290	12.69%	19.89%	271/717
One-Class SVM	KeyRecs	25.95%	—	—	—	1/99
Per-User RF	KeyRecs	18.05%	—	—	—	11/99

The per-user RF outperforms OC-SVM by 5.3 percentage points on Aalto and 7.9 points on KeyRecs. It achieves a better result than OC-SVM for 72.7% of individual users (521 out of 717). The cross-dataset comparison reveals a direct enrollment-depth effect: Aalto users with 15 sessions achieve 12.87% EER versus 18.05% for KeyRecs users with only 2 sessions under the same model — a 5.2 point improvement from 13 additional sessions. Longer enrollment sessions directly translate to better authentication accuracy.

PART III Touchscreen Gesture Dynamics

7.1 Gesture Segmentation

A gesture is defined as the sequence of touch events between a Touch Down (type=1) and the next Touch Up (type=2), including all intermediate Move events (type=0). Each gesture therefore corresponds to a single swipe, tap, or drag. The 912,133 raw events from 5 users yielded 232 valid gestures after filtering gestures shorter than 3 events (single-point taps carrying no directional information).

7.2 Feature Engineering — 19-Dimensional Gesture Vector

Temporal Features

Feature	Description
duration	Total stroke duration in seconds (Touch Down to Touch Up)
num_events	Number of touch points sampled during the stroke

Spatial Features

Feature	Description
start_x, start_y	Coordinates of Touch Down event
end_x, end_y	Coordinates of Touch Up event
length	Euclidean distance between start and end (straight-line displacement)
length_x, length_y	Horizontal and vertical displacement components

Kinematic Features

Feature	Description
speed_mean, speed_std	Average and variability of instantaneous velocity (pixels/ms)
accel_mean, accel_std	Average and variability of acceleration

Pressure Features

Feature	Description
---------	-------------

pressure_mean, pressure_std	Central tendency and variability of touch pressure
pressure_25, pressure_75	Interquartile range of pressure distribution

Direction Features

Feature	Description
direction_angle	Main stroke direction = $\arctan2(dy, dx)$, normalized to $[0, 2\pi]$
direction_change	Cumulative angular change along the stroke path — curviness vs. straightness

7.3 Model Comparison: Isolation Forest vs. One-Class SVM

Both models are trained per user using the same 70/30 genuine-data split, then evaluated against balanced genuine/impostor test sets (impostor samples drawn from the other 4 users). The key difference is how each model represents the genuine behavioral boundary:

Isolation Forest partitions the 19-dimensional feature space recursively; its boundary is implicitly defined by where the genuine data becomes sparse. It is computationally efficient and robust to irrelevant features.

One-Class SVM with RBF kernel explicitly fits a kernel-space hypersphere. The RBF bandwidth $\gamma = \text{scale}^{-1}$ adapts to the variance of each user's data, effectively normalizing the boundary tightness across users with different behavioral variability.

7.4 Results & Discussion

User	IF EER (%)	IF AUC	SVM EER (%)	SVM AUC	Best Model
0	25.00	0.7000	60.00	0.5600	Isolation Forest
1	33.33	0.7569	27.08	0.8299	One-Class SVM
2	18.52	0.8834	25.93	0.8272	Isolation Forest
3	50.00	0.6800	30.00	0.6400	One-Class SVM
4 *	16.67	0.8056	0.00	1.0000	One-Class SVM
Mean	28.70	0.7652	28.60	0.7714	—

* User 4 achieved perfect authentication: EER = 0%, AUC = 1.00 with One-Class SVM.

The near-identical mean EERs (28.70% vs 28.60%) confirm that feature quality is the dominant determinant of performance, not model choice. The large per-user variance — from EER=0% (User 4) to EER=50% (User 3) — suggests that individual users differ significantly in how distinctively they interact with the touchscreen. User 4's perfect result indicates that their pressure profile and `direction_change` signature are wholly non-overlapping with the other 4 users' gestures. User 3's 50% IF EER (equivalent to random guessing)

suggests their gesture kinematics fall squarely within the behavioral range of at least one other user; OC-SVM's explicit boundary reduces this to 30%.

8 Cross-Modality Results Summary

Modality	Algorithm	Mean EER	Best Individual EER	Users Tested
Mouse Dynamics	Isolation Forest	33.19%	—	14
Keystroke Dynamics	Per-User Random Forest	12.87%	<10% for 271/717	718
Touchscreen Gestures	One-Class SVM	28.60%	0% (User 4)	5

Keystroke dynamics outperforms the other two modalities on EER because the Random Forest benefits from explicit impostor contrast — a structural advantage that the one-class approaches cannot replicate. The touchscreen module's lower EER versus mouse (28.6% vs 33.19%) likely reflects the richer 19-dimensional feature space capturing pressure — a dimension unavailable to mouse tracking — as well as the more constrained gesture topology on a touchscreen versus free-form mouse movement.

9 Real-Time Authentication System

The combined system runs as a Flask web application with real-time scoring:

- * **Enrollment:** users type freely for 2-3 minutes (≥ 200 keystrokes) and record 3 mouse movement sessions. The backend extracts features and trains a personal OC-SVM per modality in approximately 2-3 seconds.
- * **Continuous authentication:** a keystroke sliding window scores every 20 keystrokes in real time, displaying a live color-coded histogram (green = ACCEPT, red = ALERT) with opacity fade for older windows. A moving average of the last 5 window scores smooths noise before the final verdict.
- * **Impostor detection in practice:** when a different user takes over the keyboard, the system typically flags the intrusion within 60-90 seconds (3-5 scored windows), because dwell times, flight times, and typing speed profiles diverge significantly from the enrolled model within the first few windows.

10 Limitations & Future Work

Limitation	Root Cause	Future Direction
Mouse EER 33.19%	Small cohort (14 users), behavioral overlap	50+ users; feature augmentation with click dynamics
Touchscreen EER 28.6%	Only 5 users; 232 gestures	Larger cohort; convolutional gesture encoders
Keystroke EER 12.87% vs. SOTA 3-8%	~43 windows/user on Aalto	Longer enrollment; Siamese network session similarity
Static thresholds	No adaptation to behavioral drift	Adaptive per-session threshold recalibration

Lab conditions only	No fatigue, stress, or device variation	Longitudinal field study; multi-device testing
OC-SVM $O(n^2)$ training	No impostor data for mouse/touch	Semi-supervised approach using shared impostor pool

11 Conclusion

This project demonstrated end-to-end behavioral biometric authentication pipelines across three complementary interaction modalities. The keystroke module achieved the strongest result — **12.87% EER on 718 users** — by leveraging supervised per-user Random Forest classifiers that learn explicit behavioral boundaries against impostor contrast, with dwell time emerging as the single most discriminative feature class. The touchscreen module reached a mean EER of 28.6% across 5 users and produced one case of perfect authentication (EER=0%, AUC=1.0), demonstrating that pressure-augmented gesture features carry substantial individual identity information. The mouse dynamics module achieved 33.19% EER on 14 users, with the main constraint being the small cohort size and the inherent difficulty of one-class learning without impostor contrast.

The multi-modal architecture — combining continuous keystroke and mouse behavioral verification — creates an authentication surface that goes beyond static credentials. Even if an attacker obtains a user's password, they must still replicate the enrolled user's typing rhythm and mouse kinematic profile to gain access, a task that requires not just knowledge but deep behavioral mimicry.

References

- [1] Liu, F. T., Ting, K. M., & Zhou, Z. H. (2008). Isolation Forest. *Proc. IEEE ICDM*, 413-422.
- [2] Scholkopf, B., Platt, J. C., Shawe-Taylor, J., Smola, A. J., & Williamson, R. C. (2001). Estimating the support of a high-dimensional distribution. *Neural Computation*, 13(7), 1443-1471.
- [3] Frank, M., Biedert, R., Ma, E., Martinovic, I., & Song, D. (2013). Touchalytics: On the applicability of touchscreen input as a behavioral biometric for continuous authentication. *IEEE TIFS*, 8(1), 136-148.
- [4] Feher, C., Elovici, Y., Moskovitch, R., Rokach, L., & Schclar, A. (2012). User identity verification via mouse dynamics. *Information Sciences*, 201, 19-36.
- [5] Shen, C., Cai, Z., & Guan, X. (2012). Continuous authentication for mouse dynamics: A pattern-growth approach. *Proc. IEEE/IFIP DSN*, 1-12.
- [6] Ahmed, A. A. E., & Traore, I. (2014). Biometric recognition based on free-text keystroke dynamics. *IEEE Transactions on Cybernetics*, 44(4), 458-472.
- [7] Dhakal, V., Feit, A. M., Kristensson, P. O., & Oulasvirta, A. (2018). Observations on typing from 136 million keystrokes. *Proc. ACM CHI*, 646.
- [8] Pedregosa, F. et al. (2011). Scikit-learn: Machine learning in Python. *JMLR*, 12, 2825-2830.